ASSIGNMENT 20
SQL Injection Prevention in Android Applications

1. Introduction

SQL Injection prevention requires several layers of protection. Android applications should validate user input according to the expected format, use parameterized database operations, and manage stored data and database errors securely. These practices prevent user-provided text from being interpreted as SQL syntax and reduce the chance of unintended access or modification of records.

2. Aim

To demonstrate practical techniques for preventing SQL Injection in an Android application through input validation, parameterized queries, and secure database handling.

3. Learning Objectives

- Apply allowlist validation to values received from an Android user interface.
- Replace dynamically constructed SQL statements with parameterized database operations.
- Handle cursors, database errors, and sensitive data safely.
- Verify that invalid input cannot change query behavior.

4. Tools and Setup

- Android Studio with a local Android test project.
- Android SQLiteDatabase API or SQLiteOpenHelper.
- An emulator or test device.
- A sample table containing non-sensitive dummy records.
- A code editor and test checklist.

Use a local test application and synthetic data only. Do not experiment on production databases or applications without authorization.

5. Technique One: Input Validation

Input validation checks that data follows the format expected by the application. For example, if a record ID must be a positive integer, parse it as an integer and reject empty, malformed, or out-of-range values. For a search field, set a reasonable length limit and reject invalid control characters when they are not part of the expected input.

Validation example:

```
String rawId = idField.getText().toString().trim();
int recordId;
try {
    recordId = Integer.parseInt(rawId);
    if (recordId <= 0) throw new NumberFormatException();
} catch (NumberFormatException ex) {
    showMessage("Enter a valid record ID.");
    return;
}
```

Validation improves data quality, but it is not a substitute for parameterized queries. Even apparently harmless text should not be concatenated into SQL statements.

6. Technique Two: Parameterized Queries

Parameterized queries separate the SQL command from its values. The question-mark placeholder represents a value, and the actual value is passed separately to the database.

Example using Android SQLiteDatabase:

```
Cursor cursor = db.query("notes",
    new String[] { "_id", "title" },
    "_id = ?", new String[] { String.valueOf(recordId) },
    null, null, null);
```

The value of recordId is bound as data and cannot change the SQL statement's structure. Use placeholders for every user-controlled SQL value. If a table or column name must be selected dynamically, use a fixed allowlist because SQL identifiers cannot generally be bound as value parameters.

7. Technique Three: Secure Database Handling

- Use SQLiteOpenHelper or another supported database access layer to manage database creation and upgrades consistently.
- Use ContentValues for insert and update operations instead of assembling SQL strings from user input.
- Close Cursor and other database resources reliably, for example with try/finally or a suitable resource-management pattern.
- Return generic user-facing errors; keep detailed diagnostics in controlled development logs and never log passwords or tokens.
- Store only necessary data and do not store passwords in plaintext. For authentication, use a suitable password-hashing design.

- Apply authorization checks before returning sensitive records; a parameterized query does not itself prove the caller is allowed to access a record.

8. Practical Procedure

1. Create a local table named notes with a few dummy records and a numeric _id field.
2. Identify any existing code that builds SQL by concatenating values from text fields.
3. Add input validation for the expected field format and length.
4. Replace the unsafe query with the parameterized SQLiteDatabase query shown above.
5. Test with a valid ID, an invalid ID, a non-existent ID, and a value containing SQL punctuation.
6. Confirm that the application returns only the intended record or no record, and that the input cannot change the query structure.
7. Review resource cleanup and ensure error messages do not disclose database details.

9. Observation Table

Test case	**Expected secure behavior**	**Protection applied**	**Outcome to verify**
Valid positive record ID	Only matching record is returned	Integer parsing + parameterized query	Expected record appears
Blank or non-numeric ID	Request rejected before database query	Input validation	Validation message appears
Non-existent record ID	No rows returned	Bound selection argument	No record is shown
Input containing SQL punctuation	Treated as a literal value, not SQL syntax	Parameterized query	No unexpected rows are returned
Database error condition	User sees a safe generic message	Controlled error handling	No SQL statement or secret is exposed

10. Result

Input validation, parameterized queries, and secure database-handling practices provide complementary protections against SQL Injection. In the test application, verify that valid input retrieves only the intended record, invalid input is rejected, and SQL punctuation cannot alter query logic. Record the actual results from the local test environment.

11. Precautions

- Use a local application with dummy records for testing.
- Never rely on input validation alone to prevent SQL Injection.
- Do not concatenate user-controlled values into SQL statements.
- Avoid logging credentials, tokens, personal data, or full SQL statements containing sensitive values.
- Retest the application after changes to ensure both security and normal functionality.

12. Conclusion

SQL Injection can be prevented by separating user data from SQL commands and using validation to enforce expected formats. Parameterized queries are the main defense against query manipulation, while safe resource handling, restrained logging, secure data storage, and authorization checks provide additional protection. Applying these practices together improves the security and reliability of Android database operations.

13. References

1. Android Developers documentation for SQLiteDatabase and SQLiteOpenHelper.
2. OWASP SQL Injection Prevention Cheat Sheet.
3. OWASP Mobile Application Security Testing Guide (MASTG).